

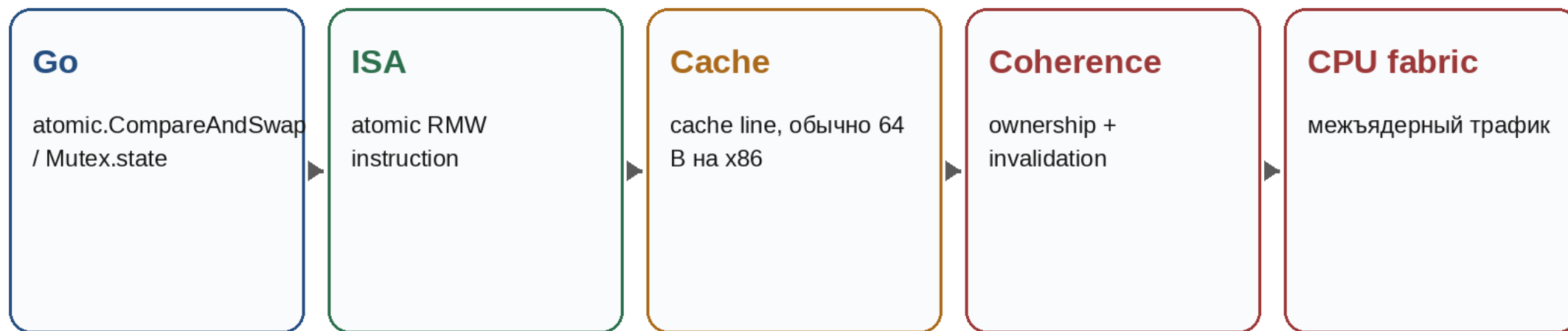
CAS, cache coherence и hot Mutex

От Go runtime к микроархитектуре CPU

- Как четыре байта `sync.Mutex.state` превращаются в межъядерную точку сериализации.
- CAS → cache line ownership → MESI/coherence → barriers/order → false sharing → production contention.

Цель: связать строку Go-кода с физическим движением ownership между CPU cores.

Одна строка Go — несколько аппаратных слоёв



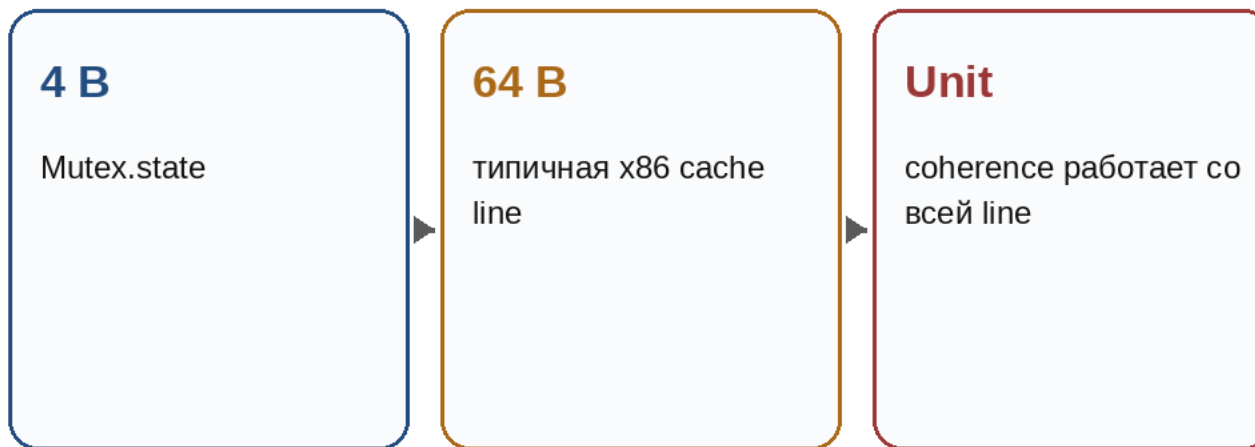
Shared mutable state имеет не только логическую, но и физическую цену.

CPU работает через иерархию памяти

- Регистры и L1/L2 значительно ближе к core, чем DRAM.
- Чтение `m.state` обычно обслуживается из cache, если нужная line уже локальна.
- Запись в память не означает немедленный физический write в DRAM.

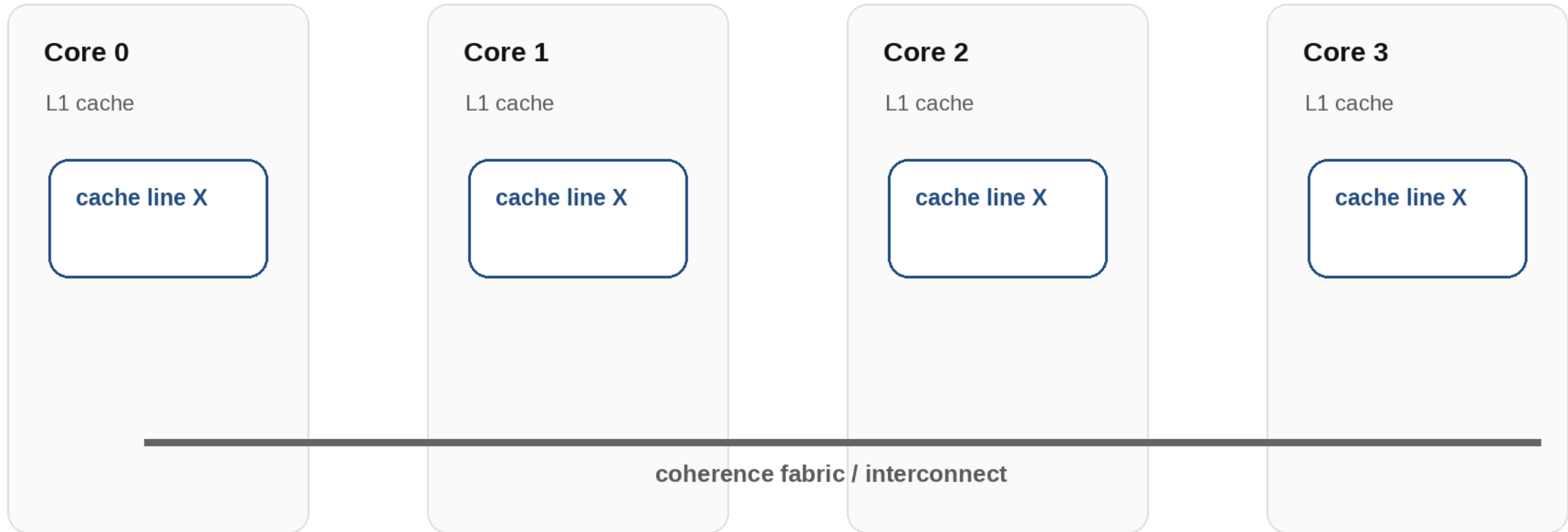
Для concurrency важнее видимость и порядок, чем «лежит ли байт уже в RAM».

Cache хранит line, а не отдельный int32



Отсюда появляется false sharing: независимые поля могут конфликтовать через одну line.

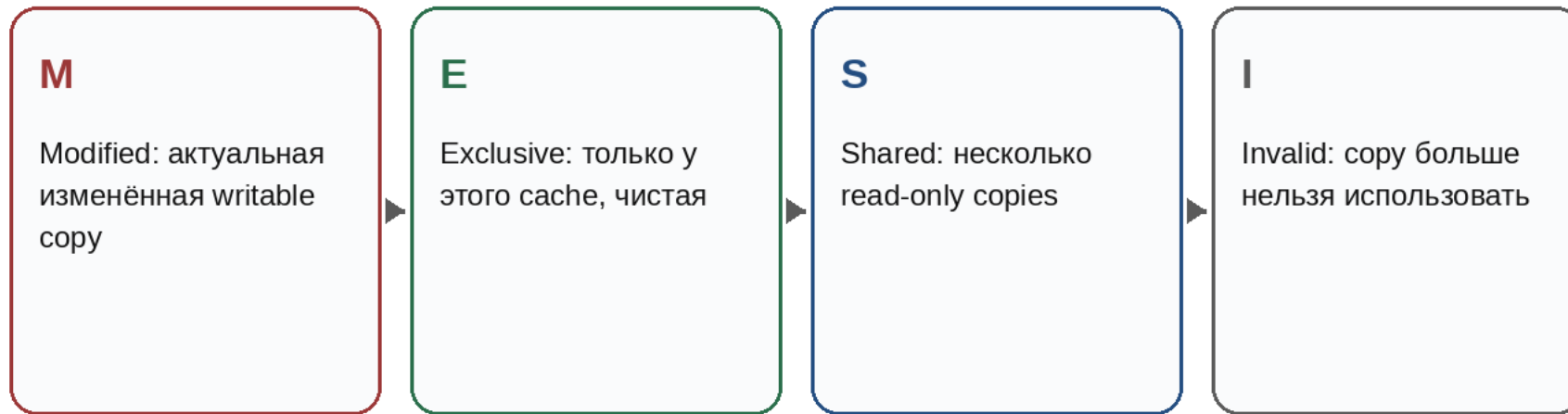
Одна line может находиться в нескольких L1



Read-only copies могут одновременно существовать на нескольких cores.

При записи одному core потребуется exclusive modification ownership.

MESI как учебная модель



Современные CPU могут использовать MESIF/MOESI и дополнительные состояния — MESI здесь ментальная модель.

Shared чтение и запись — разная стоимость

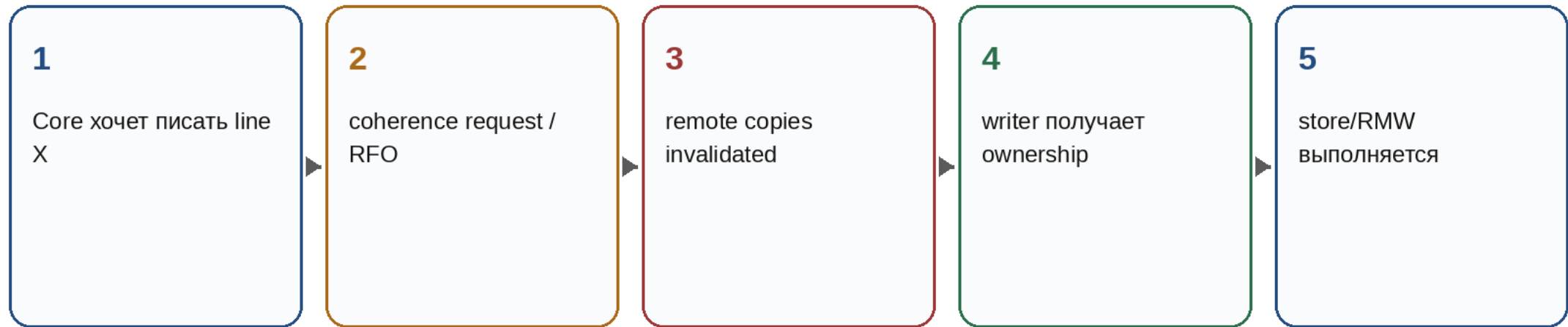
Shared read

- Core 0 и Core 1 могут иметь одну line в состоянии Shared.
- Оба читают без передачи write ownership.

Write

- Writer должен инвалидировать другие writable/readable copies согласно protocol.
- Line становится exclusive/modified у одного core.

S → M: что требуется перед записью



CAS — атомарный read-modify-write

```
if *ptr == old {  
    *ptr = new  
    return true  
}  
return false
```

- Read, compare и условная запись должны выглядеть как единое атомарное событие.
- Два cores не могут одновременно «успешно» захватить один Mutex.state.

Go CAS на amd64

```
// концептуально  
LOCK CMPXCHG [state], new
```

- LOCKed RMW синхронизируется с другими processors через cache-coherence machinery.
- Миф «LOCK всегда блокирует весь memory bus» для обычной cacheable aligned memory слишком груб.

Go CAS на ARM64

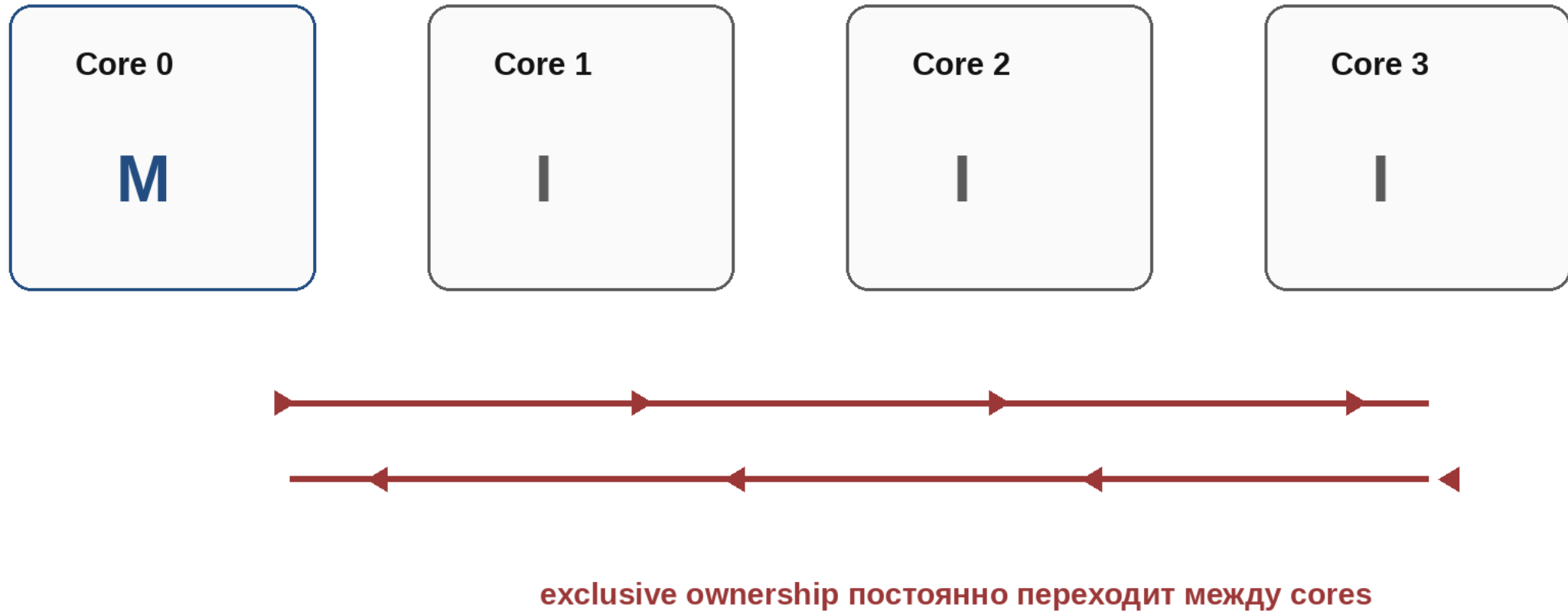
```
// LSE path
CASALW ...

// fallback
LDAXRW ...
STLXRW ...
retry on failure
```

- Instruction set отличается, семантика Go atomic API остаётся одной.
- Contention всё равно приводит к coordination за одну shared line.

Почему hot Mutex.state «летает» между ядрами

Одна writable cache line — один текущий writer owner



Cache-line ping-pong

- «Line летает» означает постоянную передачу exclusive modification ownership между cores.
- Конкретный путь может проходить через peer cache, LLC, directory и interconnect.
- Главное для performance: следующий writer ждёт coherence coordination.

Не обязательно буквальное копирование $L1 \rightarrow L1$; обязательно — смена права на модификацию.

Hot Mutex.state = global serialization point

- Все contenders модифицируют один state int32.
- Atomicity заставляет writers проходить через одну последовательность ownership transfers.
- Количество goroutines может расти, но одна line не становится многописательной.

Почему больше cores иногда не ускоряют

- Каждый дополнительный core приносит нового contender за одну shared line.
- Растут CAS failures, ownership transfers и coherence traffic.
- Throughput может выйти на плато; при тяжёлом contention — ухудшиться.

Shared-state scalability ограничена не только scheduler, но и физикой cache coherence.

CAS бывает очень разным по цене

Line уже локально writable

- Ownership уже у текущего core.
- Минимум coherence coordination.
- Uncontended atomic может быть относительно дешёвым.

Line у другого core / shared

- Нужно запросить ownership.
- Инвалидировать/согласовать remote copies.
- Latency сильно выше и зависит от topology.

Failed CAS тоже создаёт нагрузку

- Проигравший contender не получает lock, но atomic RMW всё равно участвует в coherence protocol.
- CAS-loop под contention порождает retries вокруг одной cache line.
- Lock-free означает отсутствие blocking lock в алгоритме, но не отсутствие hardware contention.

Почему бесконечный CAS-spin плох

```
for !state.CompareAndSwap(0, 1) {  
    // retry  
}
```

- Все активные cores постоянно атакуют один memory location.
- CPU занят, полезной работы почти нет.
- Cache line пинг-понгит между writers.

Зачем Mutex паркует waiters



Parking уменьшает не только CPU consumption, но и число активных contenders за hot cache line.

Coherence и memory ordering — разные задачи

Cache coherence

- Согласованность copies одного memory location/cache line.
- Кто имеет актуальную и writable copy.
- MESI/MOESI-подобные protocols.

Memory ordering

- В каком порядке разные loads/stores могут наблюдаться другими cores.
- Compiler + CPU reordering.
- Acquire/release/fence semantics.

Ordering-проблема

```
// Core 0
data = 42
ready = true

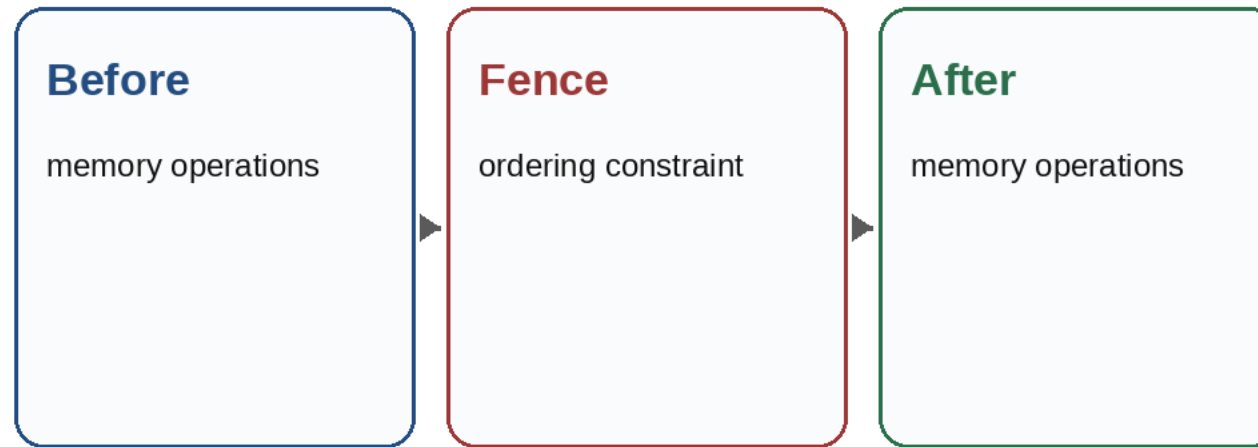
// Core 1
if ready {
    use(data)
}
```

- Без synchronization нельзя вывести «ready=true
□ data=42».
- Go memory model требует happens-before / synchronizes-before отношения.

Почему CPU переупорядочивает операции

- Out-of-order execution скрывает latency зависимостей.
- Store buffers позволяют core продолжать работу до глобальной видимости store.
- Load speculation и pipelines повышают throughput.
- Concurrency primitives должны ограничить опасные reorderings.

Memory barrier / fence



Fence не означает «вытолкнуть весь cache в RAM». Это ordering/visibility constraint.

Acquire / Release

Release

- Публикует предшествующие memory effects перед synchronization event.
- Типичный смысл: Unlock / release-store.

Acquire

- После acquire можно наблюдать опубликованные изменения.
- Типичный смысл: Lock / acquire-load.

Mutex публикует данные critical section

```
// G1
mu.Lock()
user.Name = "Alice"
user.Age = 42
mu.Unlock()

// G2
mu.Lock()
use(user.Name, user.Age)
mu.Unlock()
```

- Mutex.state — coordination point, но ordering относится ко всей правильно защищённой памяти.
- Unlock synchronizes-before последующий соответствующий Lock.

Go скрывает большую часть memory-order knobs

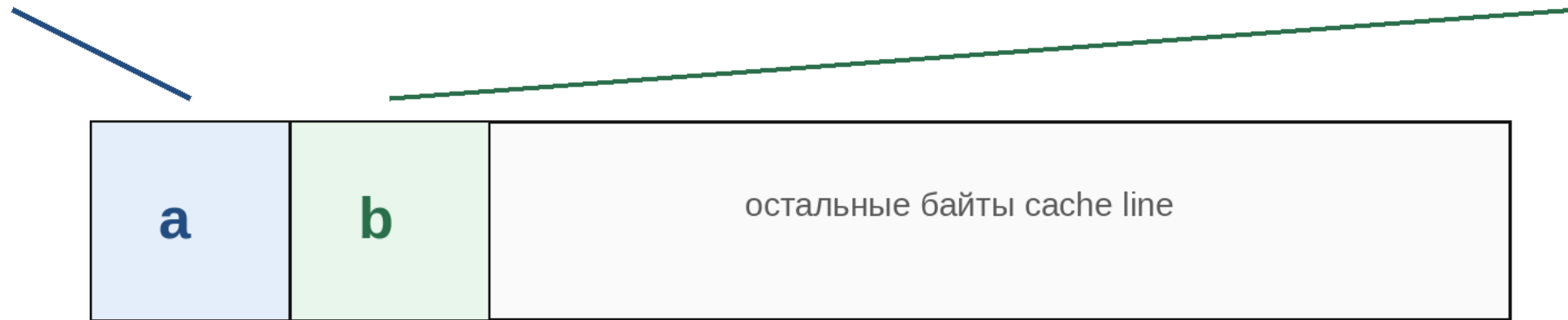
- В C++ разработчик выбирает relaxed/acquire/release/acq_rel/seq_cst.
- Публичные Go atomics имеют более простую sequentially consistent модель.
- Цена — меньше тонкой настройки; выигрыш — меньше способов написать почти правильный lock-free код.

False sharing

Логически разные переменные могут физически делить одну cache line

Core 0 обновляет a

Core 1 обновляет b



Coherence видит одну line: запись a инвалидирует core c b и наоборот.

True sharing и false sharing

True sharing

- Несколько cores реально изменяют одну переменную.
- Например один global counter или Mutex.state.
- Конфликт логический и физический.

False sharing

- Cores изменяют разные поля.
- Поля лежат в одной cache line.
- Coherence всё равно сериализует ownership line.

Почему sharding помогает физически

- Один global lock → одна hot cache line.
- 16/32 shards → несколько независимых synchronization lines.
- Снижается вероятность, что разные cores одновременно требуют ownership одной line.
- Алгоритмическая $O(1)$ может не измениться, а throughput вырасти радикально.

Production пример: global cache mutex

До

- 50k–100k RPS.
- Все handlers: Lock → map lookup → Unlock.
- Lookup быстрый, но один state постоянно меняет ownership.

После sharding

- Ключ маршрутизируется в shard.
- Несколько mutex/state cache lines.
- Меньше contention и cache-line bouncing.

Production пример: hot atomic counter

- Один `atomic.Uint64` на миллионы `updates/sec` может стать hot line.
- Per-worker/per-P counters уменьшают global write sharing.
- Периодическая aggregation меняет точность/сложность в обмен на scalability.

NUMA делает цену topology-dependent

- Передача ownership между близкими cores дешевле, чем через межсокетный interconnect.
- Один global mutex на laptop и на 128-core dual-socket server может масштабироваться совершенно по-разному.
- Atomic latency нельзя оценивать одной константой «N наносекунд».

Как доказать contention benchmark-ом

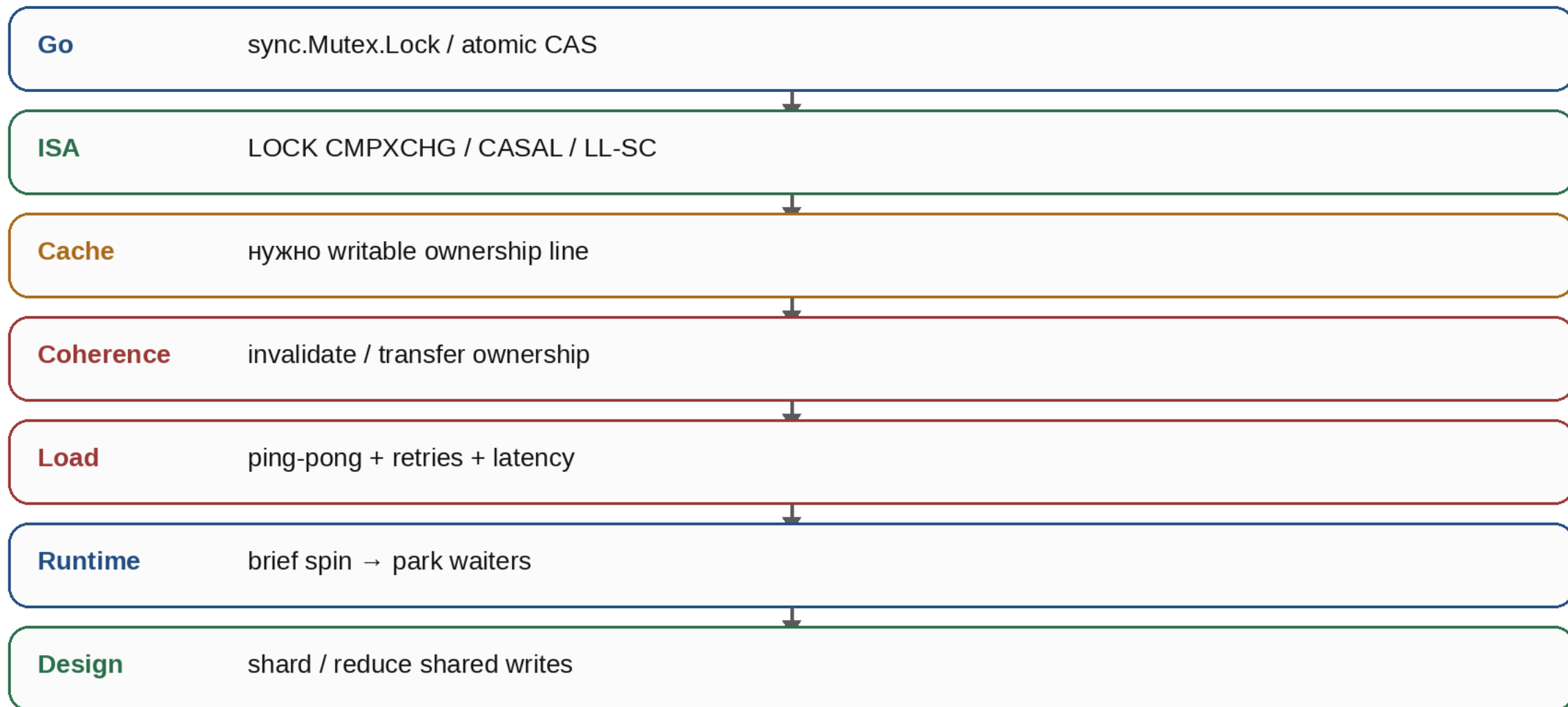
```
type Counter struct {  
    mu sync.Mutex  
    n   uint64  
}  
  
func (c *Counter) Inc() {  
    c.mu.Lock()  
    c.n++  
    c.mu.Unlock()  
}  
  
// go test -bench=. -cpu=1,2,4,8,16
```

- Смотрим scaling throughput при росте CPU.
- Потом сравниваем с sharded вариантом.
- На Linux можно дополнить perf stat / perf c2c.

Диагностика слоями

- pprof mutex/block profile: где lock contention на уровне Go.
- go tool trace: сколько G WAITING/RUNNABLE и scheduler delay.
- benchmark: подтверждаем scaling problem воспроизводимо.
- perf/PMU: при необходимости спускаемся к cache-to-cache sharing и hardware events.

Полная причинно-следственная цепочка



Что важно запомнить

- CAS — atomic RMW; его стоимость зависит от состояния и местоположения cache line.
- Coherence и ordering решают разные задачи.
- Hot shared writer создаёт ownership ping-pong и физическую сериализацию.
- Parking уменьшает активный CAS/coherence pressure.
- False sharing создаёт hardware contention даже без логического shared variable.
- Sharding масштабируется потому, что распределяет mutable state по независимым lines.

Shared mutable state имеет физическую цену в cache-coherence fabric.

Источники для углубления

- Go Memory Model — go.dev/ref/mem
- sync и sync/atomic documentation — pkg.go.dev/sync, pkg.go.dev/sync/atomic
- Go runtime atomic implementations — [internal/runtime/atomic](https://internal.go.dev/runtime/atomic)
- Intel® 64 and IA-32 Architectures Software Developer's Manual
- Arm Architecture Reference Manual / atomic & memory-ordering sections